

bitfence: a pre-transaction risk layer for autonomous on-chain agents

Dennis Wecker, bitfence

Abstract

Autonomous AI agents execute on-chain transactions without a human in the loop. The risk-assessment tooling in production today was built for a human reader. bitfence is a pre-transaction risk oracle for autonomous agents operating on Solana, Base, Ethereum, Arbitrum, and BSC, with a sixth chain (HyperEVM) implemented and gated behind a live release check. It decomposes token risk into six categories, each a distinct mechanism by which a holder's position is destroyed — exit integrity, supply integrity, liquidity integrity, holder and insider risk, provenance, and market integrity — and assembles a weighted composite with live-category renormalization, maturity-aware scoring, and circuit breakers surfaced as structured response fields. Scoring runs in a pure, versioned policy engine with an explicit evidence discipline: every fact feeding a circuit breaker is three-valued (known-true, known-false, unknown), breakers fire on known evidence only, cross-source disagreement marks a fact contested rather than silently averaging it, and an assessment that could not settle a safety-critical check is delivered with a named degradation and a recommendation capped at human approval. Facts are drawn from a shared primary provider across all chains, a per-chain RPC leg that includes bitfence's own transaction-simulation honeypot measurement on EVM chains, and free advisory cross-validators. The same assessment contract is exposed through three interfaces: an HTTP/JSON API, a hosted MCP server, and Rust-native bindings for the Rig agent toolkit. Access and settlement happen per call in USDC micropayments on Base, implemented against the Coinbase CDP facilitator. Releases are gated by a live-token check that measures every chain's data legs before it ships enabled; the confidence value reported with every assessment reflects only the sources actually consulted, and known structural gaps are named rather than papered over.

1. The problem: risk assessment for autonomous on-chain agents

Autonomous agents now execute on-chain transactions without a human in the loop at the moment of execution. Agent frameworks — Rig, ElizaOS, Coinbase AgentKit, MCP tool-calling harnesses — let a language model hold a wallet, read a mandate, and broadcast trades. The operator composes the mandate and reviews the report; the agent is the actor that touches the chain.

Risk-assessment tooling in production today was built for a human reader. Block explorers render transaction histories into scannable tables. Web dashboards surface categorical labels — “honeypot,” “retained ownership,” “high tax” — in panels meant for a trader reviewing an unfamiliar token. Browser wallet warnings intervene when a person is about to confirm a prompt. An agent does not scan, does not review, and does not confirm; it consumes structured fields and routes on them.

The mismatch is interface, not capability. Re-packaging a dashboard behind a REST wrapper does not close it: the response shape (prose plus categorical labels), the billing model (per-account API keys), and the latency tolerance (seconds in a browser tab) all reflect the human audience those tools were designed for.

Risk assessment for an autonomous agent must complete before the transaction broadcasts; an agent cannot unwind a settled transaction. After-the-fact monitoring serves reconciliation and operator telemetry; it does not protect the next trade. The check must run inline, within the latency budget of a single tool call.

A risk layer for autonomous agents must satisfy four requirements:

1. **Interface.** Callable inline as a tool, returning a response shape consistent across chains and integration paths, so a tool-calling model can route on fields rather than interpret prose.
 2. **Economics.** Priced per call without account creation or conventional billing.
 3. **Interpretability.** A score, an actionable recommendation (PROCEED, REQUIRE_HUMAN_APPROVAL, BLOCK), and structured hard-safety flags surfaced as explicit response fields, not folded into a composite.
 4. **Fail-safe degradation.** A human reviewing a dashboard notices when a panel is blank; an agent consuming a score does not. When an upstream data source is unavailable, the layer must say so explicitly and must not let the outage silently disable the one check that would have caught a scam token.
-

2. Scoring methodology

Risk scores are integers on a 0–100 scale, where 0 is minimal risk and 100 is maximum risk.

The six categories

Each category is one distinct mechanism by which an agent’s position is destroyed. Signals live under the mechanism they evidence, never under the data source or technique that produced them — machine learning is a technique, not a category, and a data vendor is a supplier, not a mechanism.

1. **Exit Integrity (25%).** Can a holder sell — now, and will they still be able to later? Honeypot behaviour, sell and transfer tax levels, tax modifiability, freeze authority, and transfer-blocking configuration. Total loss; the most common trap.
2. **Supply Integrity (20%).** Can supply or balances be manipulated at the contract level — minted, confiscated, frozen, or the contract destroyed? Mint authority, hidden ownership, owner-editable balances, permanent delegates, ownership recovery, upgradeable proxies, SELFDESTRUCT, metadata mutability. Total loss via dilution or confiscation.
3. **Liquidity Integrity (20%).** Can the market itself be withdrawn? LP lock and burn status, the unlock schedule, pool depth, deployer-held LP, pool age. Total loss via rug.
4. **Holder & Insider Risk (15%).** Who holds the supply, and will they dump on you? Top-holder concentration, deployer share, holder count, sniper/insider/bundler cohorts, and the age distribution of holder wallets. Partial loss.
5. **Provenance (12%).** Who made this token, and what is their record? Deployer track record, observed prior rugs, vendor scam verdicts, launchpad lifecycle, verification registries. A prior rather than a direct mechanism — but deployer track record is among the most predictive rug signals in the surveyed literature.
6. **Market Integrity (8%).** Is the observed activity real? Wash-trade ratios, the share of old-wallet flow on young tokens, and price/volume divergence. Manufactured volume and momentum are a deception mechanism aimed specifically at agents that trade on those signals.

The category weights order the mechanisms by expected severity of loss, with the two total-loss trap mechanisms — blocked exit and withdrawn liquidity — and contract-level supply manipulation together carrying 65% of the composite.

Composite with live-category renormalization

The composite is a weighted average across the six categories. A category is live when its evaluator returns at least one signal. A category that returns no signals — because a data source was unavailable, an adapter call failed, or the fact does not exist for the chain — drops out of both the numerator and denominator; its nominal weight is redistributed proportionally across the live categories. Within a category, the category score is the maximum of its signal sub-scores: the single worst signal determines the category’s contribution.

Renormalization ensures that an absent category neither pulls the composite toward “safe” nor anchors it to an arbitrary midpoint. The composite reflects only the categories for which real data was obtained. Whether an absent category makes the *assessment* unsafe to act on is a separate question, answered by the evidence discipline below rather than by the score.

Evidence discipline

Every fact that feeds a circuit breaker is three-valued: known-true, known-false, or unknown. A source returning a negative is knowledge — an indexer answering “no pairs” is a known absence of a pool. A fetch error is not — it leaves the fact unknown. An unknown is never defaulted into a known, in either direction: missing data can neither fire a breaker nor silently suppress one.

A breaker fires only when every fact it needs is known. When a fact a breaker needs is unknown, the breaker is *indeterminate*, and the assessment is delivered as **degraded**: the response names the unavailable sources explicitly, and the recommendation is capped at `REQUIRE_HUMAN_APPROVAL` — it can never be `PROCEED`. The score itself stays fail-open (dark categories drop out and weights renormalize over live ones), but the recommendation fails safe. An upstream outage cannot cause the system to wave a token through on the strength of the checks it happened to skip.

The cap follows indeterminate breakers, not dark sources. If a source is down but every breaker still reaches a known conclusion from the remaining evidence — for example, authority state is known-clean, so an authority breaker is settled regardless of what the dark source would have said — no check was disabled and no cap applies. Degradation is reported when it exists and enforced when it matters.

Because most facts have more than one source, the discipline extends across sources. Agreement between sources adds nothing beyond the fact being grounded; disagreement beyond a per-fact tolerance marks the fact **contested** — it drops out of confidence grounding and is named in the response reasoning. A breaker predicate fires on *any* source’s known value, not only the primary’s: a threat measured by a cross-validator is still a measured threat.

Trusted-registry tokens (below) are exempt from the degradation cap: the registry entry is curated evidence covering what the dark source would have proven.

Circuit breakers

Circuit breakers are safety overrides evaluated after the composite. A breaker can raise the score to a forced minimum but cannot lower it: a composite already above a breaker’s floor is never reduced. All breakers are evaluated independently; the final score is the maximum of the applicable floors, and every fired breaker is named in the response.

Every breaker belongs to one of two classes, and the classification is part of the breaker’s definition, not a call-site judgment:

Measured-threat breakers fire on direct measurement of an active threat — the token demonstrably cannot be sold, contains a self-destruct path, or carries a confiscatory measured sell tax. They override everything, including trusted-token status: “a major stablecoin that measurably cannot be sold” means a wrong address or a live compromise, and must block.

1. **Confirmed honeypot** — the sell leg of bitfence’s own transaction simulation reverts.
2. **SELFDESTRUCT opcode** detected in the contract bytecode.
3. **High sell tax** above a blocking threshold, measured rather than claimed.

Heuristic breakers fire on risk *inference* — authority configuration, holder concentration, LP status, a vendor’s rug verdict. They are strong evidence of danger for unknown tokens, and they floor the score for any token outside the trusted registry. They never override trusted-token status, because the same configuration can be legitimate by design: Circle holds mint and freeze authority on USDC deliberately.

4. **Hidden owner and mint** — mint and freeze authority both active on a non-established Solana token, with an EVM analogue for hidden ownership combined with mintability.
5. **Owner-editable balances** — the owner can rewrite holder balances; confiscation capability.
6. **Owner-modifiable taxes** — the tax is acceptable today and the owner can raise it after entry.
7. **LP fully unlocked and deployer-held** — the deployer can withdraw liquidity instantly.
8. **Imminent LP unlock** — a material share of locked liquidity unlocks within days; “locked until tomorrow” stops scoring like “locked for a year.”
9. **Already rugged** — a vendor-observed rug on this token. Heuristic despite being an observed outcome: it is a vendor’s verdict, not bitfence’s own measurement, and must not override trusted-token status on a detector glitch.
10. **Extreme holder concentration** — top holders control a disproportionate share.
11. **Insider-cohort concentration** — sniper, insider, and bundler cohorts together hold a majority of supply.
12. **Deployer concentration on a young pool** — a classic rug-setup pattern.
13. **No DEX pool with very few holders** — no discoverable liquidity and thin distribution.
14. **Micro holder count** on a non-mature token.

The final score and recommendation resolve in a fixed precedence order, highest first: measured-threat breakers, then the trusted-token cap, then heuristic breakers, then the degraded-assessment cap, then the composite.

Trigger thresholds and forced-minimum floors are not published. Publishing them would let adversarial deployers engineer tokens that sit immediately below the trigger points. The response tells the calling agent which breakers fired; it does not tell a deployer where the trigger points sit.

Maturity-aware scoring

A naive model penalises long-lived tokens for the same characteristics a new scam token exhibits — retained mint authority, unlocked LP, a held tax switch. Three mechanisms address the maturity asymmetry between long-lived and new tokens:

1. **Trusted-token registry** (curated). A single curated registry of established tokens — major stablecoins, wrapped assets, and well-known governance tokens — with one semantics: a registry token’s score is capped in the Low band, it is exempt from heuristic breakers and from the degraded-assessment cap, and it is never exempt from measured-threat breakers. The registry prevents false positives on tokens like Wrapped SOL, where retained mint authority is a design feature rather than a rug setup, without creating a class of token the system will vouch for unconditionally.
2. **Established tokens** (computed, softer test). When the token’s primary pool is known to have existed beyond a maturity threshold, authority and tax-modifiability sub-scores are downgraded and the establishment-conditioned heuristic breakers are disabled.
3. **Mature tokens** (computed, stricter test). Known evidence of long survival — a pool older than a stricter threshold or a holder base above a large-distribution threshold — exempts a token from the LP-related heuristic breakers.

Computed relief is granted on known evidence only. A token whose pool age cannot be determined is neither established nor unestablished — the fact is unknown, a breaker that needs it goes indeterminate, and the assessment degrades. Missing data never buys an exemption from a threat that known facts established, and never manufactures a threat on its own. Thresholds are not documented, for the same reason breaker trigger points are not.

Confidence

Confidence is a measure of coverage, not a measure of how correct the score is. It reports the fraction of the chain’s source-bearing units that fully answered, uncontested, during the assessment.

For each chain, the data sources that exist for that chain are grouped into units; a unit is grounded when its sources answered and agree within tolerance. A known negative is an answer: “this token has no DEX pool” grounds the pool unit as completely as pool data would. Sources that do not exist for a chain are excluded from both numerator and denominator — Solana’s confidence is not permanently depressed by the absence of a honeypot simulation, and an optional source that is not configured reads as narrower coverage by design, not as a failed fetch. A fully grounded assessment reaches 1.0. A transient upstream failure, or a contested fact, reduces confidence for that request without affecting the system’s nominal coverage.

One chain carries a structural discount: every HyperEVM assessment’s confidence is multiplied by 0.8. A HIP-1-linked token’s real market may live on the HyperCore orderbook rather than in the on-EVM pools bitfence reads, so liquidity and market-integrity signals there can be distorted; until linked-asset detection exists, every chain-999 assessment carries the discount uniformly, preserving ordering between assessments.

Confidence is reporting, not enforcement. The safety consequence of missing data is enforced server-side by the degraded-assessment cap described above: a caller does not need to interpret a float correctly to stay safe. Confidence remains useful as a graded signal — an operator can log it, alert on it, or set a stricter policy than the server enforces.

3. Chain coverage and data sources

bitfence supports five chains in production — Solana, Base, Ethereum, Arbitrum, and BSC. A sixth adapter, HyperEVM, is implemented and currently gated: every chain ships enabled

only after passing a live-token release check (Section 4), and HyperEVM has not yet passed its confidence bar. The choice of chains reflects where agent-driven trading activity currently concentrates.

The provider stack has three tiers, and the tier a fact arrives from never changes what the fact means:

1. **A shared primary provider.** A single commercial market-data API serves all chains from one adapter: liquidity depth and lock schedules, holder concentration and cohort statistics, deployer track record, launchpad lifecycle, verification registries, and the per-bucket market-activity series behind the market-integrity signals. One round trip per assessment; parity across chains is measured, not assumed.
2. **A per-chain RPC leg, breaker-grade.** On Solana: mint-account parsing (including Token-2022 TLV extensions read directly, without a crate dependency), freeze and mint authority, metadata mutability. On the EVM chains, a JSON-RPC endpoint per chain drives bytecode and storage probes (SELFDESTRUCT scan with a PUSH-aware opcode walker, proxy detection), LP-burn checks against dead addresses, deployer detection, and — the decisive exit facts — **bitfence’s own honeypot and trade-tax measurement:** an `eth_call` transaction simulation that routes a buy and a sell through the chain’s DEX router and observes whether and at what cost the sell completes. Honeypot status on EVM chains is measured by bitfence, not purchased as a vendor verdict.
3. **Free advisory cross-validators.** Independent public token-security and market-data services corroborate and contest facts from the first two tiers. They can ground a fact the primary missed, contest a fact it got wrong, and fire a breaker on a known threat — but they are advisory in the operational sense: a rate-limited co-source cannot veto an assessment or a release, only widen or narrow its confidence.

Category	Primary	RPC leg	Advisory
Exit Integrity	—	Solana: authority + Token-2022 fee parsing. EVM: <code>eth_call</code> honeypot/tax simulation	tax and transfer-blocking configuration
Supply Integrity	mintability	Solana authority parsing; EVM bytecode/proxy probes	ownership facts (incl. hidden owner, EVM)
Liquidity Integrity	liquidity, locks, unlock schedule	EVM LP-burn dead-address checks	LP holder breakdown, pool corroboration
Holder & Insider	holders, cohorts, wallet ages	Solana holder fallback	holder counts
Provenance	creator record, launchpad, verification	deployer detection	creator history, rug verdicts
Market Integrity	activity buckets	—	market corroboration

The specific vendors behind each tier are an operational detail, deliberately undocumented here: facts carry provenance internally, and a provider can be swapped without changing what any fact means or how any signal scores.

Structural gaps are named rather than papered over. No honeypot simulation exists for Solana anywhere in the surveyed field; the fact stays unknown there and degradation reporting handles it. HyperEVM has no vendor security facts from anyone; its supply and exit facts exist only as bitfence's self-computed probes, which is one reason its confidence is discounted. Sanctions and blocklist screening is not carried by any surveyed provider and is outside the current taxonomy.

Adapters return typed facts from a concurrent fan-out with hard per-call deadlines; every fact records which sources observed it, and the policy engine consumes facts directly, so every fact in an assessment is either grounded in a source that answered or explicitly unknown because one did not. The scoring layer is chain-agnostic: adding an EVM chain the primary provider and co-sources already cover is a configuration row and an RPC endpoint, not a change to the composite, circuit breakers, or response shape.

4. Architecture and integration

bitfence is a single Rust binary exposing one scoring engine through three interfaces: an HTTP/JSON API, an MCP (Model Context Protocol) server, and Rust-native bindings for the Rig agent toolkit. Each interface returns the same `RiskAssessment` structure. Adding a fourth interface means an additional adapter at the edge delegating to the same analyzer; it does not mean a rewrite.

Tool-calling LLM agents — Claude Desktop, MCP clients embedded in IDEs, hosted MCP harnesses — speak MCP natively; a hosted MCP endpoint removes an integration step. Rust-native agents built on Rig bind a typed in-process tool, not an HTTP call. Everything else — scripts, scheduled jobs, other-language frameworks — uses the HTTP API. Standardising the scoring engine rather than the transport is the integration-surface claim.

Internally, a Tower middleware stack handles tracing, CORS, per-IP rate limiting, and body limits. An analyzer function performs validation, cache lookup, the fan-out to data sources, the policy verdict, and the cache write; adapters are the only layer that performs network I/O for chain data. The fan-out runs as one bounded wave with a hard deadline per source call, so a stalled vendor degrades one fact rather than the request. Cached responses return in single-digit milliseconds; a cold assessment typically completes in a few hundred milliseconds.

All scoring decisions — composite assembly, evidence grading, breaker evaluation, precedence resolution — live in a pure, versioned policy engine with no I/O of its own. The policy carries an explicit version, and every assessment is stamped with the version that produced it. Because the policy is pure over typed inputs, the full precedence behaviour — including outage scenarios that are impractical to reproduce against live chains — is exercised in deterministic tests.

Releases are gated the same way facts are: by measurement. Before a release ships, a live-token check runs a fixed panel of real tokens per chain from the production host, asserting facts and per-leg latency. Only breaker-grade legs — the primary provider and the chain's RPC leg — can fail a chain; a chain that fails ships disabled behind its kill switch while the release deploys on schedule, and enables when its legs pass. This is how HyperEVM is currently gated.

A two-tier cache — moka L1 in-process, Redis L2 optional — sits in front of the analyzer. Cache TTLs are keyed to the final risk score rather than to token maturity or adapter source. The rationale is adversarial: if an upstream data source is briefly compromised or manipulated and produces a distorted score, the window during which that score persists in cache is bounded. High-risk false positives are re-evaluated quickly; low-risk scores can safely persist longer. Three

further rules keep the cache honest: every cached envelope stores the facts that produced it, and a warm hit re-derives the verdict from those facts under the running policy version; a cached entry stamped with an older policy version is treated as a miss, so no cached verdict outlives the policy that produced it; and degraded assessments are never written to the shared cache, so an upstream outage is re-checked per request rather than being served as if it were a settled answer. Exact durations are not published.

x402 is implemented against the Coinbase CDP facilitator for USDC micropayments on Base. A request without payment receives a 402 **Payment Required** response whose body carries the x402 challenge plus Bazaar discovery metadata — name, description, JSON-Schema, example input and output — for the Coinbase CDP marketplace. The caller’s client signs a payment and retries the request with a signed payment header; the server verifies the payment via a CDP endpoint with EdDSA JWT bearer authentication, runs the handler, and settles on-chain only when the handler produced a successful response — a failed assessment is never charged. If settlement fails, the response is discarded and a 502 is returned; a paid response is never delivered to a non-paying client. A constant-time API-key bypass is available for partner integrations, comparing against a configured key without timing-oracle leaks.

Contextual risk

The `POST /v1/risk/contextual` endpoint layers position-specific analysis on top of the base assessment. Given a token address, a position size in USD, and the caller’s total portfolio size in USD, it returns the base assessment plus a `context` object with the following fields:

1. **Pool liquidity** in the token’s primary DEX pool.
2. **Estimated slippage** for the specified position size.
3. **MEV exposure category** — low, medium, or high — based on position-size-to-pool-liquidity ratio, calibrated to the sandwich-attack patterns in the prior literature.
4. **Portfolio concentration** — the position as a percentage of the caller’s total portfolio.
5. **Safe position size** — the largest position size the endpoint treats as within its safe-slippage band, calculated from the pool’s liquidity depth.
6. **Recommended position cap**, considering both slippage and portfolio concentration.

The endpoint can escalate a recommendation but cannot lower it. A `BLOCK` remains `BLOCK` regardless of position size; a `PROCEED` escalates to `REQUIRE_HUMAN_APPROVAL` when estimated slippage or portfolio concentration cross escalation thresholds.

5. Current scope and extensions

The live system today comprises six signal categories on five chains exposed through three interfaces (HTTP, MCP, Rig), with a sixth chain implemented and gated. The extensions below are research directions, not dated commitments.

HyperEVM enablement is the nearest concrete step. The adapter, probes, and simulation exist and pass their health legs; the chain is held by its confidence bar — three sources instead of five, plus the structural 0.8 discount described in Section 2. It enables when an additional data leg or linked-asset detection lifts grounded confidence over the release floor, with no code change to the scoring engine.

Linked-asset detection on HyperEVM is the research direction behind that discount: detecting when an HIP-1 token's canonical market lives on the HyperCore orderbook, so on-EVM liquidity facts can be marked as partial views rather than discounted wholesale.

Async fact arrival is an architectural extension already reflected in the evidence discipline: advisory co-sources that answer after the cold-path deadline can ground or contest facts for the next assessment of the same token, narrowing the gap between what is knowable and what is knowable within one latency budget.

Sanctions and blocklist screening is a named gap: no surveyed provider carries it, and it stays outside the taxonomy until a dedicated provider meets the evidence bar. Additional chains follow the adapter pattern: an EVM chain that the primary provider and co-sources already cover is a configuration row and an RPC endpoint.

The service is listed on Coinbase Bazaar and ClawHub. An ElizaOS plugin is in development.

ethereum/ERCs PR #1638 proposes ERC-8211 ("Smart Batching"), a draft standard for composing multi-step on-chain actions into a single signed intent. If the proposal merges, it becomes a natural upstream point for agents binding multiple on-chain actions, including a pre-transaction risk check, into one signed transaction.

6. Related work and positioning

Three services operate in the adjacent space: GoPlus, RugCheck, and honeypot.is.

GoPlus is a multi-chain token risk API and dashboard that returns categorical risk labels (honeypot, tax, owner privileges, liquidity health, and related fields) to human developers and end-users via REST and a web UI. Its surface shape and account model assume developer integration into human-facing applications.

RugCheck is a Solana-focused token analysis service that surfaces LP holder composition, contract authority state, and community-reported flags through a web UI and public endpoints. The service is single-chain and browser-first, with consumption patterns that follow from that audience.

honeypot.is is a simulation-based EVM service that detects tokens whose sell path reverts, along with estimated buy and sell taxes, returned as a single JSON record per token. The scope is category-specific; bitfence runs its own transaction simulation for this signal family rather than consuming a vendor verdict, so the measurement carries first-party provenance.

bitfence differs from these along five architectural dimensions:

1. It runs before the transaction rather than after.
2. The same assessment contract is exposed through three interfaces, with MCP and Rig binding agent frameworks natively.
3. Circuit breakers are structured, first-class response fields, not dashboard alerts.
4. It degrades fail-safe: an upstream outage that leaves a safety check unresolved caps the recommendation at human approval and is named in the response, rather than silently weakening the assessment.
5. Access and settlement happen per call in USDC on Base rather than per account via API keys.

References

1. **x402 protocol specification.** Coinbase, coinbase/x402 repository (<https://github.com/coinbase/x402>); CDP facilitator documentation (<https://docs.cdp.coinbase.com/x402/>)
2. **Model Context Protocol specification.** Anthropic. <https://spec.modelcontextprotocol.io/>
3. **ERC-8211: Smart Batching.** ethereum/ERCs pull request #1638. Draft, pending merge. <https://github.com/ethereum/ERCs/pull/1638>
4. **Xia et al.** “Trade or Trick? Detecting and Characterizing Scam Tokens on Uniswap Decentralized Exchange.” *ACM Internet Measurement Conference (IMC)*, 2021.
5. **Mazorra, Adan, Daza.** “Do Not Rug on Me: Leveraging Machine Learning Techniques for Automated Scam Detection.” *Mathematics*, vol. 10, no. 6, 2022.
6. **Cernera et al.** “Token Spammers, Rug Pulls, and Sniper Bots: An Analysis of the Ecosystem of Tokens in Ethereum and in the Binance Smart Chain (BNB).” *USENIX Security Symposium*, 2023.
7. **Daian et al.** “Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges.” *IEEE Symposium on Security and Privacy*, 2020.